ReconForge — Canonical Terminology Reference

> **Purpose:** Single source of truth for all naming conventions, identifiers, and terminology used across ReconForge documentation and code.
> Last updated: 2026-03-21

Modules

Canonical Name	CLI Subcommand	Python Class	Directory
Network	`network`	`NetworkModule`	`modules/network/`
Web	`web`	`WebModule`	`modules/web/`
API	`api`	`APIModule`	`modules/api/`
Surface	`surface`	`SurfaceModule`	`modules/surface/`
AD	`ad`	`ADModule`	`modules/ad/`
Workflow	`workflow`	`WorkflowOrchestrator`	`core/workflow_orchestrator.py`

> Always capitalize module names when referring to them as proper nouns (e.g., “the Network module”, “the AD module”).

Phase Slugs (CLI `--phases` Values)

Network Module

CLI Slug	Internal Phase Class	Description
`discovery`	`HostDiscoveryPhase`	Ping sweep, ARP scan, live host identification
`scanning`	`PortScanningPhase`	SYN/connect scan, port enumeration
`enumeration`	`ServiceEnumerationPhase`	SMB, LDAP, deep service analysis
`authentication`	`AuthenticationChecksPhase`	Anonymous access, credential testing, hydra (opt-in)

Web Module

CLI Slug	Internal Phase Class	Description
`surface`	`SurfaceDiscoveryPhase`	Technology fingerprinting, WAF detection
`content`	`ContentEnumerationPhase`	Directory/file brute-forcing
`vuln`	`VulnerabilityScanningPhase`	Nuclei scanning, vulnerability identification
`exploit`	`ExploitCandidatesPhase`	SQLMap, exploit identification (**opt-in**)

API Module

CLI Slug	Internal Phase Class	Description
`discovery`	`DiscoveryPhase`	Endpoint discovery, OpenAPI parsing
`authentication`	`AuthenticationPhase`	JWT analysis, auth mechanism testing
`fuzzing`	`FuzzingPhase`	Parameter fuzzing, error-based discovery
`authorization`	`AuthorizationPhase`	IDOR/BOLA testing, privilege checks (**opt-in**)

Surface Module

CLI Slug	Internal Phase Class	Description
`port_discovery`	`PortDiscoveryPhase`	Stealth port scanning
`service_fingerprint`	`ServiceFingerprintPhase`	Service identification and normalization
`vector_correlation`	`VectorCorrelationPhase`	Cross-service correlation
`prioritization`	`PrioritizationPhase`	Attack vector ranking

AD Module

CLI Slug	Internal Phase Class	Description
`passive`	`PassiveReconPhase`	Anonymous LDAP/SMB enumeration
`identity`	`IdentityEnumerationPhase`	User/group enumeration
`configuration`	`ConfigurationEnumeration-Phase`	GPO, trust, delegation analysis
`delegation`	`DelegationDiscoveryPhase`	Kerberos delegation discovery
`bloodhound`	`BloodhoundCollectionPhase`	BloodHound data collection

CLI Flags

Flag	Modules	Description
`-t, --target`	All	Target specification
`--opsec`	All	OPSEC mode: `stealth`, `normal`, `aggressive`
`--phases`	All	Comma-separated phase slugs
`-o, --output`	All	Output base directory
`-v, --verbose`	All	Verbose output
`--dry-run`	All	Show commands without executing
`--timeout`	All	Global timeout in seconds (default: 600)
`--encrypt-loot`	Network, Web, API, AD, Workflow	Encrypt loot with Fernet (**not on Surface CLI**)
`--brute-force`	Network	Enable hydra brute-force (opt-in)
`--domain`	AD	Active Directory domain name
`-u, --username`	AD	AD username
`-p, --password`	AD	AD password
`--dc-ip`	AD	Domain controller IP
`--wordlist, -w`	Web, API	Custom wordlist path
`--threads, -th`	Web	Thread count (default: 40)
`--extensions, -e`	Web	File extensions to fuzz
`--follow-redirects`	Web	Follow HTTP redirects (default: true)
`--verify-ssl`	Web	Verify SSL certificates
`--header`	API	Custom HTTP headers (repeatable)
`--auth-token`	API	Authentication token

Flag	Modules	Description
`--modules`	Workflow	Comma-separated module list
`--engagement`	Workflow	Engagement name
`--client`	Workflow	Client name
`--operator`	Workflow	Operator name
`--resume`	Workflow	Path to saved engagement JSON

OPSEC Modes

Mode	Description
`stealth`	Low-noise techniques, reduced scan rates, minimal fingerprinting
`normal`	Balanced approach (default)
`aggressive`	Full-speed scanning, all techniques enabled

Severity Levels

Ordered from most to least severe. Used in `FindingsManager.add()`.

Level	Description
`critical`	Confirmed exploitable, high-impact vulnerability
`high`	Strong evidence of exploitability with significant impact
`medium`	Moderate evidence or moderate impact
`low`	Weak evidence or minimal impact
`info`	Informational, no direct security impact

Confidence Levels

Ordered from most to least confident. Used in `FindingsManager.add()`.

Level	Max Allowed Severity	Description
`confirmed`	`critical`	Exploited or verified vulnerability
`high`	`critical`	Strong evidence of exploitability
`medium`	`high`	Moderate evidence requiring further validation
`low`	`medium`	Weak evidence requiring manual verification
`heuristic`	`low`	Pattern-based detection with no concrete evidence

> **Severity clamping:** `FindingsManager` automatically caps severity based on confidence. A `heuristic` finding is never rated above `low` severity.

Finding Types

Used in `FindingsManager.add()` as the `finding_type` parameter:

Type	Description
`vulnerability`	Exploitable security weakness
`misconfiguration`	Insecure configuration
`exposure`	Unintended information disclosure
`credential`	Discovered or weak credentials
`attack_vector`	Identified attack path
`information`	General informational finding
`assessment`	Assessment-level observation
`prioritisation`	Attack surface prioritization result

Exception Classes

All defined in `core/exceptions.py`:

Exception	Parent	Description
`ReconForgeError`	`Exception`	Base exception for all ReconForge errors
`ConfigError`	`ReconForgeError`	Configuration loading/parsing errors
`ProfileNotFoundError`	`ConfigError`	Requested profile not found
`ValidationError`	`ReconForgeError`	Input validation failures
`TargetValidationError`	`ValidationError`	Invalid target specification
`PortValidationError`	`ValidationError`	Invalid port specification
`ExecutionError`	`ReconForgeError`	Tool execution failures
`ToolNotFoundError`	`ExecutionError`	Required external tool not installed
`TimeoutError`	`ExecutionError`	Tool exceeded timeout
`ModuleError`	`ReconForgeError`	Module-level errors
`PhaseError`	`ModuleError`	Phase execution errors
`WorkflowError`	`ReconForgeError`	Workflow orchestration errors
`WorkflowAbortedError`	`WorkflowError`	Workflow aborted by user or condition
`CredentialVaultError`	`ReconForgeError`	Credential vault operations
`EngagementError`	`ReconForgeError`	Engagement management errors
`EngagementNotFoundError`	`EngagementError`	Engagement file not found

Core Components

Component	File	Description
`Runner`	`core/runner.py`	Subprocess execution with timeout, logging, OPSEC
`FindingsManager`	`core/findings_manager.py`	Finding creation, severity clamping, deduplication
`LootManager`	`core/loot_manager.py`	Loot file storage with optional Fernet encryption
`CredentialVault`	`core/credential_vault.py`	Credential storage with deduplication and encryption
`NotesManager`	`core/notes_manager.py`	Operator notes and annotations
`OutputManager`	`core/output_manager.py`	Directory structure, JSON/Markdown/HTML output
`ConfigLoader`	`core/config_loader.py`	YAML configuration loading
`ProfileLoader`	`core/profile_loader.py`	OPSEC profile resolution from `profiles.yaml`
`ToolConfig`	`core/tool_config.py`	Tool path/argument resolution from `tools.yaml`
`EngagementManager`	`core/engagement.py`	Engagement lifecycle (start/pause/resume/complete)
`OpsecChecker`	`core/opsec_checks.py`	OPSEC risk assessment for commands
`WorkflowOrchestrator`	`core/workflow_orchestrator.py`	Cross-module workflow execution
`AttackWorkflow`	`core/attack_workflow.py`	Kill chain tracking and hypothesis management
`DetectionMap`	`core/detection_map.py`	Tool noise-level mapping for OPSEC decisions
`TargetParser`	`core/target_parser.py`	Target string parsing and validation

Configuration Files

File	Description
`config/tools.yaml`	Tool paths, default arguments, timeout overrides
`config/profiles.yaml`	OPSEC profiles defining phase/tool restrictions

Output Structure

```
outputs/<target>/
├── <module>/
│   ├── findings.json          # Structured findings
│   ├── <module>_summary.md    # Human-readable summary
│   ├── <module>_summary.html  # HTML report
│   ├── raw/                   # Raw tool output
│   └── loot/                  # Extracted loot files
└── ...
```

Naming Conventions in Documentation

Context	Convention	Example
Module names	Capitalized	“the Network module”, “the AD module”
Phase slugs	Lowercase, as used in CLI	`discovery`, `scanning`
CLI flags	Exact syntax with `--` prefix	`--encrypt-loot`, `--opsec`
Exception names	PascalCase, exact class name	`TimeoutError`, `ValidationError`
Severity levels	Lowercase	`critical`, `high`, `medium`, `low`, `info`
Confidence levels	Lowercase	`confirmed`, `high`, `medium`, `low`, `heuristic`
Python classes	PascalCase	`FindingsManager`, `NetworkModule`
Config keys	snake_case	`encrypt_loot`, `brute_force`